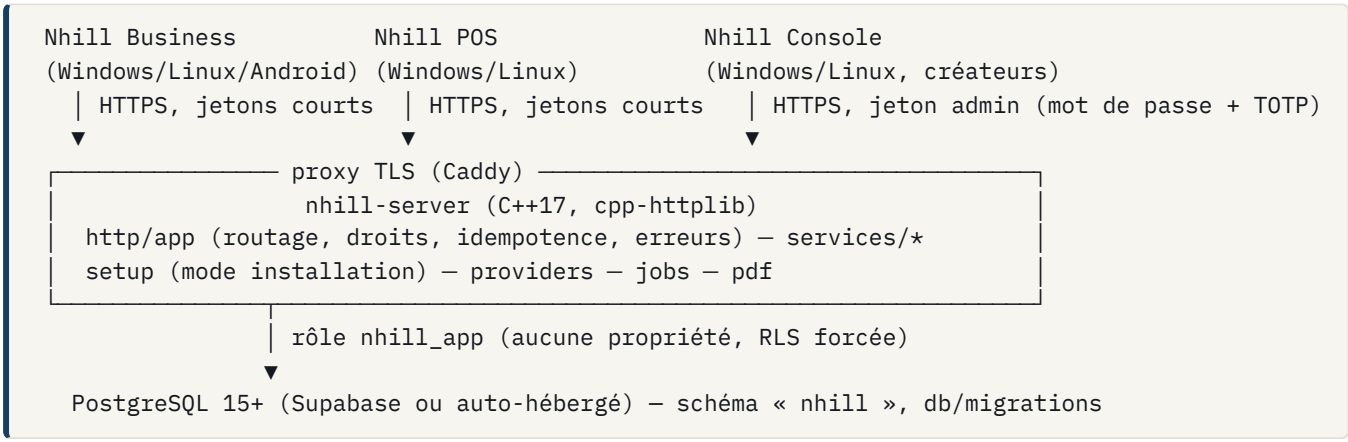
# Nhill 2.1 — Guide du développeur

*English: ../en/developer-guide.md*

Pour comprendre le code, le modifier sans casser ses garanties et le faire vérifier par la CI. Référence détaillée de l'architecture et des règles métier : `docs/technique.md` ; API : `docs/api.md`.

## 1. Architecture en bref




Invariants à ne jamais rompre :

- le client ne contient **aucun identifiant de base de données** et ne parle qu'à l'API ;
- chaque requête est une transaction ; `Tx::setTenant` fixe `app.establishment_id` et la base filtre elle-même les tables métier (`FORCE ROW LEVEL SECURITY` sur 30 tables) ;
- les montants sont des entiers en unités mineures (`common/` : `Money`), jamais des nombres à virgule ; les devises ne sont jamais additionnées entre elles ;
- un document émis est immuable ; on corrige par avoir ;
- le journal d'activité est en ajout seul (aucune route ne le modifie) ;
- les opérations d'écriture sensibles sont idempotentes (`Idempotency-Key`).

## 2. Arborescence

| Dossier | Contenu |
|---|---|
| `common/` | bibliothèque C++ partagée serveur/clients : montants, codes d'établissement, validations (dont la règle des mots de passe), version |
| `server/src/` | `http/` (application, routage), `services/` (domaines), `setup.*`, `jobs/`, `pdf/`, `migrate.*`, `db.*`, `config.*` |
| `server/profiles/`, `server/i18n/` | profils fiscaux pays, libellés FR/EN du serveur (PDF, journal) |
| `db/migrations/` | `0001` … `0011`, empreintes vérifiées au démarrage ; une migration publiée ne se modifie jamais |
| `client/src/` | `core/` (ApiClient, Session, Validator, AppContext), `business/`, `console/`, `pos/` |
| `client/qml/Nhill/` | `Ui/` (système de conception), `Business/`, `Console/`, `Pos/` |
| `client/i18n/` | `nhill_fr.ts`, `nhill_en.ts` |
| `tests/` | `common/`, `server/` (PostgreSQL jetable), `client/`, `support/minigtest/` |

| Dossier | Contenu |
|---|---|
| `packaging/` | Debian (`linux/`), Windows (NSIS, MinGW), Android |
| `scripts/` | construction, vérification, livraison, QA, i18n, développement |
| `website/` | site vitrine et backend des formulaires (indépendants de l'application) |
| `docs/`, `legal/`, `marketing/` | documentation FR/EN (Markdown = source, PDF générés) |

## 3. Poste de développement

```
scripts/install-deps-linux.sh --with-packaging --with-docs --yes   # Debian 13 / Ubuntu 24.04
scripts/build-linux.sh                                             # compile + tests
NH_DEV_STATE=/tmp/nhdev scripts/dev/dev-stack.sh start build       # PostgreSQL jetable + API
127.0.0.1:8080
python3 scripts/dev/seed-demo.py --email demo@example.com           # données de DÉMONSTRATION
NHILL_SERVER=http://127.0.0.1:8080 build/client/nhill-business
NH_DEV_STATE=/tmp/nhdev scripts/dev/dev-stack.sh stop
```

- `dev-stack.sh` trouve PostgreSQL dans `/usr/lib/postgresql/<version>/bin` (la plus récente) ; `NH_PG_BIN` force un autre dossier.
- Console d'administration locale : `build/server/nhill-server --config /tmp/nhdev/server.env admin-bootstrap admin@exemple.dev "Admin"`, puis « Premier accès » dans `build/client/nhill-console`.
- Les données de démonstration sont toutes marquées « (démo) » ; le script refuse un serveur non local.
- **C++17 obligatoire** : libpqxx et cpp-httplib de la distribution sont compilés en C++17 ; compiler Nhill en C++20 rompt l'ABI.

## 4. Tests

| Suite | Commande | Nombre (2.1.0) | Dépendances |
|---|---|---|---|
| `common` | `ctest -R common` ou `scripts/test-common-standalone.sh` | 22 | aucune (le script autonome utilise `tests/support/minigtest`) |
| `server` | `ctest -R server` | 100 (dont 16 pour le mot de passe oublié, avec un faux relais SMTP local) | PostgreSQL 15+ local (`initdb`, `pg_ctl`) |
| `setup` | `ctest -R setup` | 6 | PostgreSQL |
| `client` | `ctest -R client` | 13 | Qt 6 |
| `keychain` | `ctest -R keychain` (stockage réel : `dbus-run-session` + `gnome-keyring-daemon` sous Linux ; `NHILL_KEYCHAIN_REQUIRED=1` pour échouer au lieu d'ignorer) | 3 (écriture et relecture dans deux processus, déconnexion, sans « Se souvenir de moi ») | Qt 6, QtKeychain, un trousseau du système |
| Site : backend | `cd website/backend && python3 -m unittest discover -s tests` | 13 | Python 3 (SQLite) ; PostgreSQL en CI |
| QML (statique) | `python3 scripts/qa/qml-check.py` | 98 fichiers | Node.js |

| Suite | Commande | Nombre (2.1.0) | Dépendances |
|---|---|---|---|
| Site (statique) | `python3 website/tools/check_site.py website` | 24 pages | Python 3 |

Les tests serveur démarrent un PostgreSQL jetable ; en root ils utilisent l'utilisateur `postgres`. Un test qui dépend des limites de débit appelle `resetIpThrottles()` d'abord.

**Règle** : toute modification d'une règle métier ou de sécurité s'accompagne d'un test qui échoue avant la modification et réussit après.

## 5. Interface (QML) : système de conception

Composants de `Nhill.Ui`, à utiliser au lieu de contrôles Qt nus :

| Composant | Usage |
|---|---|
| `Theme` | jetons : couleurs (clair/sombre, contrastes WCAG AA vérifiés), espacements `s1…s6`, tailles `fs_xs` (12) … `fs_xl`, rayons, `fitWidth()` / `fitHeight()` pour dimensionner une fenêtre selon l'écran |
| `FbButton` | variantes `primary`, `secondary`, `ghost`, `danger` ; `tip` = info-bulle (obligatoire pour un bouton à icône seule) ; anneau de focus |
| `FbTextField` | `echoMode: TextInput.Password` ajoute le bouton afficher/masquer ; `accessibleName` |
| `Field` | libellé + aide + erreur (`error`) ; donne le nom accessible au champ |
| `DataTable` | colonnes avec `minWidth`, défilement horizontal, info-bulles, navigation clavier |
| `Notifier` | messages (`Nav.toast`) ; erreurs 8 s, 4 au maximum, rôle `AlertMessage` |
| `Icon`, `Banner`, `FbCard`, `FbComboBox`, `FbSwitch`, `DateField`, `TabStrip` | voir les fichiers |

Règles : aucune couleur ni taille en dur dans les pages (jetons du thème) ; tout texte visible passe par `qsTr()` ; toute fenêtre utilise `Theme.fitWidth/fitHeight` et un minimum borné par l'écran ; tester en 1366×768, 1280×720, 390×844 (téléphone) et en thème sombre.

Contrôle statique : `python3 scripts/qa/qml-check.py` (syntaxe JavaScript des gestionnaires, fichiers listés dans les CMakeLists, textes non traduits). Captures sans écran : `scripts/qa/ui-screenshots.sh` (Xvfb).

## 6. Traductions

Les sources sont en anglais (`qsTr("…")`). Après ajout de textes :

```
lupdate client -ts client/i18n/nhill_fr.ts client/i18n/nhill_en.ts   # outil Qt
python3 scripts/i18n/ts-set.py traductions.json                      # remplit les traductions
FR depuis un JSON
```

Libellés du serveur (PDF, journal d'activité) : `server/i18n/fr.json` et `en.json` ; chaque clé d'action `action.*` doit exister dans les deux.

## 7. Base de données et migrations

- Nouvelle migration = nouveau fichier `db/migrations/00NN_nom.sql`. **Ne jamais modifier une migration publiée** : le serveur compare les empreintes SHA-256 et refuse de démarrer.
- Toute nouvelle table métier : colonne `establishment_id`, `ENABLE` + `FORCE ROW LEVEL SECURITY`, politique `USING (establishment_id = (SELECT nh_current_establishment()))` (modèle : `0009_pos.sql`), droits minimaux pour `nhill_app`, index des clés étrangères ; ajouter un test d'isolation.
- Le mode installation vérifie l'isolation par 10 contrôles : en ajouter un si la nouvelle table porte des données sensibles.

## 8. API

- Préfixe `/api/v1` ; une évolution incompatible impose `/api/v2` (et `v1` servie pendant la transition). `GET /api/v1/meta` publie `min_client_version`.
- Erreurs : `{"error": {"code": …, "message": …, "request_id": …}}` (plus `field` pour une saisie refusée) ; les codes sont traduits côté client (`AppContext`). Ajouter le message FR/EN de tout nouveau code.
- Entrées analysées strictement (`services/common`), tailles bornées ; jamais de SQL construit par concaténation (paramètres `$1…`).

## 9. Sécurité dans le code

- Mots de passe : `nhill::validation::checkPassword` (8 caractères minimum, liste de mots de passe courants) côté client **et** serveur ; hachage Argon2id (`crypto::hashPassword`).
- Comparaisons de secrets en temps constant (`crypto::constantTimeEquals`).
- Secrets des prestataires scellés avec une sous-clé de `NH_SECRET_KEY`.
- Rien de personnel dans les journaux ; erreurs client masquées côté serveur.
- Voir securite.md et l'audit 03-audit-securite.md.

## 10. Intégration continue

| Workflow | Contenu |
|---|---|
| `ci.yml` | compilation, tests (common, server, setup, client), contrôle QML, captures d'écran |
| `linux-packages.yml` | `.deb` pour Ubuntu 24.04 et Debian 13 dans des conteneurs propres + installation de contrôle |
| `windows.yml` | Qt 6.5.3 MSVC, installateurs NSIS, installation silencieuse de contrôle |
| `android.yml` | SDK/NDK, Qt pour Android, APK |
| `docs-website.yml` | PDF à jour, site vitrine, tests du backend (SQLite et PostgreSQL) |

Les artefacts de la CI sont ceux à publier ; ne publiez jamais un binaire construit sur un poste personnel non tracé.

## 11. Version

Une seule source : `project(nhill VERSION x.y.z)` dans `CMakeLists.txt`. Les scripts, les paquets, l'installateur Windows et l'APK la lisent. À mettre à jour aussi : `common/include/nhill/version.hpp` (`kProductVersion`), `QT_ANDROID_VERSION_CODE`, ressources `.rc` Windows, journaux des modifications FR/EN.